



Security-Controlled Infrastructure Deployment with **Terraform & Conftest**

"What if insecure infrastructure could be blocked before it ever reached AWS?"

• @Daniel Rr 



IaC Can Scale Problems Too

Terraform makes infrastructure repeatable, consistent, and automated.

But that same automation can just as easily repeat mistakes — at scale.

- ✗ Publicly exposed SSH
- ✗ Excessive IAM permissions
- ✗ Missing encryption

 **Goal: catch insecure infrastructure before it ever deploys**

• @Daniel Rr 



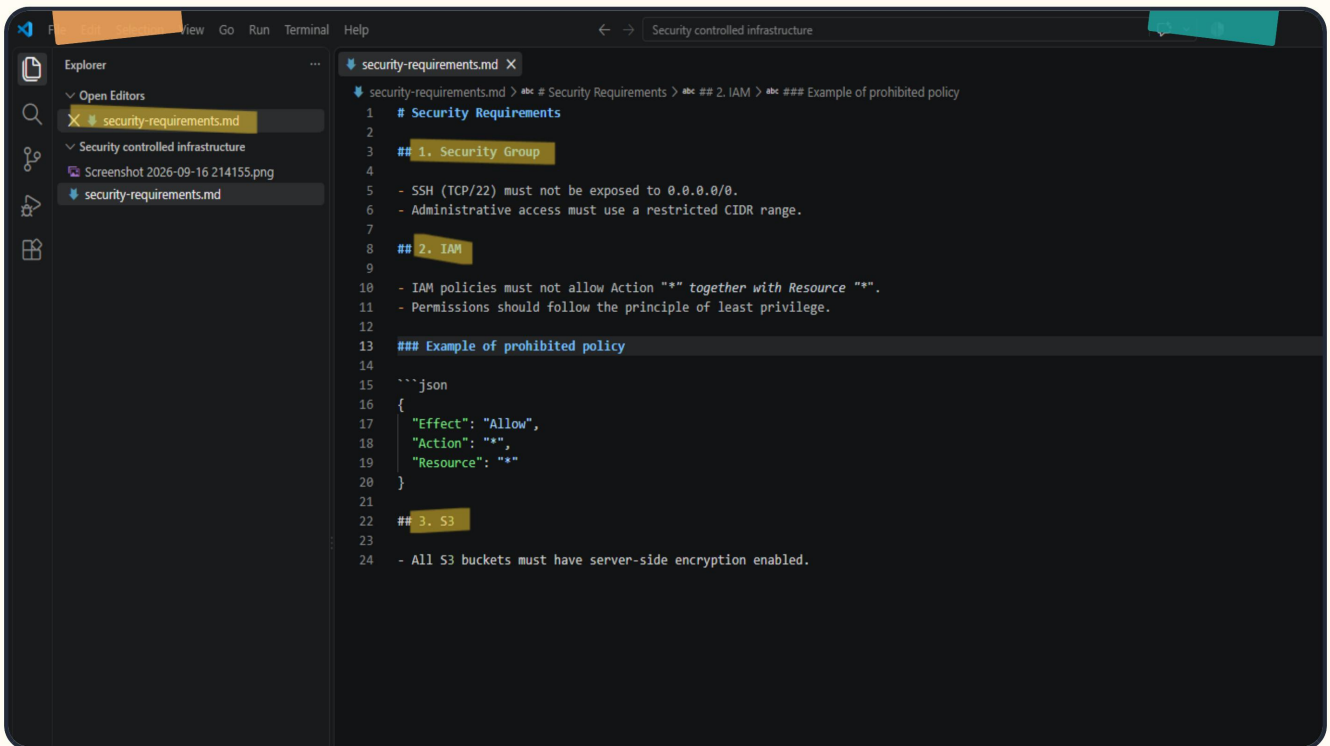
Define Security Requirements

Before writing a single policy, I locked in three non-negotiables:

 Security Group → block SSH from 0.0.0.0/0

 IAM → block Action="*" + Resource="*"

 S3 → require server-side encryption



The screenshot shows a code editor with a file named `security-requirements.md` open. The file content is as follows:

```
1 # Security Requirements
2
3 ## 1. Security Group
4
5 - SSH (TCP/22) must not be exposed to 0.0.0.0/0.
6 - Administrative access must use a restricted CIDR range.
7
8 ## 2. IAM
9
10 - IAM policies must not allow Action "*" together with Resource "*".
11 - Permissions should follow the principle of least privilege.
12
13 ### Example of prohibited policy
14
15 ```json
16 {
17   "Effect": "Allow",
18   "Action": "*",
19   "Resource": "*"
20 }
21
22 ## 3. S3
23
24 - All S3 buckets must have server-side encryption enabled.
```

• @Daniel Rr 

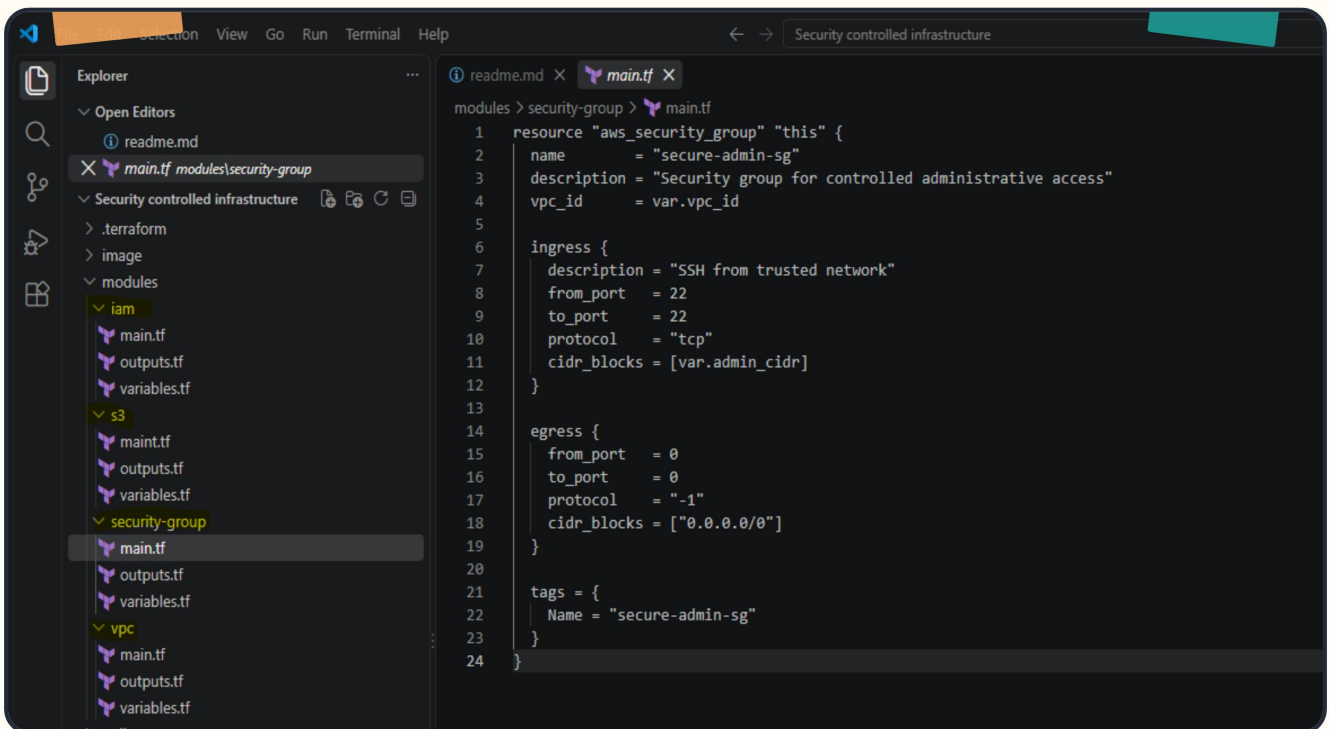


Build Reusable Terraform Modules

Instead of one giant file, everything was split into modules:

- VPC
- Security Group
- IAM
- S3

Each ships with main.tf, variables.tf, outputs.tf — security rules baked right in.



The screenshot shows a VS Code editor with a Terraform project. The Explorer on the left shows a directory structure with modules for iam, s3, security-group, and vpc. Each module contains main.tf, outputs.tf, and variables.tf files. The main.tf file for the security-group module is open in the editor, showing the following Terraform code:

```
1 resource "aws_security_group" "this" {
2   name       = "secure-admin-sg"
3   description = "Security group for controlled administrative access"
4   vpc_id     = var.vpc_id
5
6   ingress {
7     description = "SSH from trusted network"
8     from_port   = 22
9     to_port     = 22
10    protocol    = "tcp"
11    cidr_blocks = [var.admin_cidr]
12  }
13
14  egress {
15    from_port   = 0
16    to_port     = 0
17    protocol    = "-1"
18    cidr_blocks = ["0.0.0.0/0"]
19  }
20
21  tags = {
22    Name = "secure-admin-sg"
23  }
24 }
```

• @Daniel Rr 



Deploy the Infrastructure

Modules were composed from the root config to provision real AWS resources:

- VPC
- Security Group
- IAM Role & Policy
- S3 Bucket + Encryption

Before applying anything, I reviewed the full Terraform plan first.

```
To perform exactly these actions, run the following command to apply:
  terraform apply "tfplan"
PS G:\Cloud things\Project\Security controlled infrastructure> terraform apply tfplan
module.s3.aws_s3_bucket.this: Creating...
module.s3.aws_s3_bucket.this: Creation complete after 2s [id=security-controlled-iac-bali-2026]
module.s3.aws_s3_bucket_server_side_encryption_configuration.this: Creating...
module.iam.aws_iam_role_policy.this: Creating...
module.s3.aws_s3_bucket_server_side_encryption_configuration.this: Creation complete after 1s [id=security-controlled-iac-bali-2026]
module.iam.aws_iam_role_policy.this: Creation complete after 2s [id=security-controlled-role:restricted-s3-access]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

bucket_arn = "arn:aws:s3::security-controlled-iac-bali-2026"
iam_role_arn = "arn:aws:iam::031949580850:role/security-controlled-role"
security_group_id = "sg-05b90ca633891d09c"
vpc_id = "vpc-07371677d94ab53a2"
PS G:\Cloud things\Project\Security controlled infrastructure> terraform output
bucket_arn = "arn:aws:s3::security-controlled-iac-bali-2026"
iam_role_arn = "arn:aws:iam::031949580850:role/security-controlled-role"
security_group_id = "sg-05b90ca633891d09c"
vpc_id = "vpc-07371677d94ab53a2"
PS G:\Cloud things\Project\Security controlled infrastructure> |
```

• @Daniel Rr 

Policy as Code

I used Conftest + OPA/Rego to turn the rules into executable policies:

security_group.rego

iam.rego

s3.rego

The plan was converted to tfplan.json and evaluated automatically.

✓ 3 / 3 tests passed

```
PS G:\Cloud things\Project\Security controlled infrastructure>
PS G:\Cloud things\Project\Security controlled infrastructure>
PS G:\Cloud things\Project\Security controlled infrastructure>
PS G:\Cloud things\Project\Security controlled infrastructure>
PS G:\Cloud things\Project\Security controlled infrastructure>
PS G:\Cloud things\Project\Security controlled infrastructure>
PS G:\Cloud things\Project\Security controlled infrastructure>
PS G:\Cloud things\Project\Security controlled infrastructure>
PS G:\Cloud things\Project\Security controlled infrastructure>
PS G:\Cloud things\Project\Security controlled infrastructure> conftest test tfplan.
```

```
3 tests, 3 passed, 0 warnings, 0 failures, 0 exceptions
```

```
PS G:\Cloud things\Project\Security controlled infrastructure> |
```

• @Daniel Rr 



Break It On Purpose

To prove the policies worked, I intentionally shipped bad configs:

- SSH open to 0.0.0.0/0
- IAM Action="*" + Resource="*"
- S3 encryption removed

FAIL → Security violation detected

```
PS G:\Cloud things\Project\Security controlled infrastructure> cmd /c "terraform show -json tfplan > tfplan.json"
PS G:\Cloud things\Project\Security controlled infrastructure>
PS G:\Cloud things\Project\Security controlled infrastructure> (Get-Content tfplan.json -Raw | ConvertFrom-Json).resource_
changes |
>> Where-Object { $_.type -eq "aws_security_group" } |
>> ForEach-Object { $_.change.after.ingress }

cidr_blocks      : {0.0.0.0/0}
description      : SSH from trusted network
from_port        : 22
ipv6_cidr_blocks : {}
prefix_list_ids  : {}
protocol         : tcp
security_groups  : {}
self             : False
to_port          : 22

PS G:\Cloud things\Project\Security controlled infrastructure> conftest test tfplan.json --policy policy --all-namespaces
FAIL - tfplan.json - terraform.security_group - SSH (TCP/22) must not be exposed to 0.0.0.0/0

3 tests, 2 passed, 0 warnings, 1 failure, 0 exceptions
PS G:\Cloud things\Project\Security controlled infrastructure> |
```

• @Daniel Rr 



The Final Workflow

- Terraform Plan
- ConfTest Security Check
- 3 Tests Passed 
- Terraform Apply

Only an approved, policy-checked plan is ever allowed to reach AWS.

Apply complete! 

```
3 tests, 3 passed, 0 warnings, 0 failures, 0 exceptions
PS G:\Cloud things\Project\Security controlled infrastructure> terraform apply tfplan

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

bucket_arn = "arn:aws:s3:::security-controlled-iac-bali-2026"
iam_role_arn = "arn:aws:iam::031949580850:role/security-controlled-role"
security_group_id = "sg-05b90ca633891d09c"
vpc_id = "vpc-07371677d94ab53a2"
PS G:\Cloud things\Project\Security controlled infrastructure> terraform state list
module.iam.aws_iam_role.this
module.iam.aws_iam_role_policy.this
module.s3.aws_s3_bucket.this
module.s3.aws_s3_bucket_server_side_encryption_configuration.this
module.security_group.aws_security_group.this
module.vpc.aws_vpc.this
PS G:\Cloud things\Project\Security controlled infrastructure> |
```

Lessons Learned

- ① Reusable modules can standardize security
- ② Policy as Code makes security testable
- ③ Negative testing matters just as much as positive
- ④ Plan validation should always happen before Apply

Detect after deployment → Prevent before deployment



*Found this useful?
Follow for more hands-on
AWS & DevSecOps breakdowns*

@yourhandle